Data Visualisation Week 6: Visualising qualitative data

What's happening today?

Today we have an ambitious goal of taking our open text response data from our survey during the lecture and turning it into an informative data visualisation in the form of word clouds and bar charts. There is a lot of code in this handout, but most of it is building off of previous steps (so there will be a lot of copying and pasting, and not as much typing!).

Getting started

We are going to be using a whole host of new packages today because we will be working with qualitative text data, which is a whole subfield of data science and machine learning. We're also going to be doing a couple of new and ambitious things like reading our data via Google Sheets, so that the data we are analysing is newly updated when new responses are received (this would be considered a 'Data Pipeline').

```
install.packages(c("googlesheets4", "tidytext", "gutenbergr", "textdata", "ggwordclou
d", "localLLM"))
```

Now let's load all those packages along with the tidyverse.

```
library(tidyverse)
library(googlesheets4)
library(tidytext)
library(gutenbergr)
library(textdata)
library(ggwordcloud)
library(localLLM)
```

Accessing our Google Sheets Data

Today we are going to use data from the survey that we completed in the lecture on Tuesday. That data was collected through a Google Form and has been stored in a Google Sheet which is automatically updated. I will show you what that Google Sheet looks like in the workshop.

For now, access to our Google Sheet is restricted to people with a University of Sheffield Google Account. In order to access it, we need a way to tell R that **we have permission to view the sheet**. Luckily, the googlesheets4 package makes that very easy.

If you run the following code you will be asked to log into Google and provide permission for the Tidyverse (meaning on your computer) to read and write Google Sheets that are accessible to your account.

```
gs4_auth()
```

Make sure you select your Sheffield Google Account and not your personal one (or when prompted, use your @sheffield.ac.uk email address), then click continue, then check the box with the text "See, edit, create and delete all your Google Sheets spreadsheets." and click continue again. Your R script will now be authorised to access Google Sheets that have been created by or shared with you.

Let's try reading in our data. I have created a bit.ly link to reduce the amount of typing you need to do, but I have to use a bit of magic first in order for it to work.

```
url <- httr::GET("https://bit.ly/edc117survey")$url
read_sheet(url)
```

The first line of code asked R to get the web address that the short link from bit.ly is pointing to, and then saves it in an object called url that we can use instead of having to type out "https://docs.google.com/spreadsheets/d/1y6Ep8KZcmTY_Itso3p5Lbw6QPo2Viy0zaXkQrCFO5Bo/edit?usp=sharing (https://docs.google.com/spreadsheets/d/1y6Ep8KZcmTY_Itso3p5Lbw6QPo2Viy0zaXkQrCFO5Bo/edit?usp=sharing)"

This looks okay, but the variable names are way too long, we definitely don't want to have to be typing them all out ourselves. Luckily, we can give R a vector of shortened names we want to use. However, because we are now ignoring our original headers, we also want to tell googlesheets4 that the first row of our data is no longer necessary by using skip = 1. Let's add this and see the result.

```
survey <- read_sheet(url,
                     skip = 1,
                     col_names = c("time", "resident", "feel_before",
                                   "know_before", "feel_now", "perfect_day")
                     )

glimpse(survey)
```

Working with qualitative text data: Topics versus Sentiment

There are two things we are potentially interested in visualising with our qualitative data: topics, that is, the things people are talking about, and sentiment, that is, how people feel about something. Each of these require slightly different steps to process the data.

Preparing data for text analysis

The first thing we need to do when we are preparing long-form qualitative data for visualisation or other forms of quantitative analysis is put it in a form that our computer can use — these are called 'tokens'. This means **one or two words** per row. We can do this using the unnest_tokens() function from tidytext. When doing this, it is best to focus on one free-text variable at a time using select(). Let's start with the responses to the 'perfect day' question:

```
survey %>%
  select(time, resident, perfect_day) %>%
  unnest_tokens(word, perfect_day)
```

The first argument in unnest_tokens() is the name of the new, tokenised column we are creating. The second argument is the name of the variable with the original responses.

Now that we have tokenised the responses, we probably want to remove words that do not really provide any information about the topic; these can be a lot of 'articles' like the, a, an, but also some other common verbs or adjectives.

```
survey %>%
  select(time, resident, perfect_day) %>%
  unnest_tokens(word, perfect_day) %>%
  anti_join(stop_words)
```

Okay, now we can visualise the data.

Visualising topics in text data

The most straightforward thing we can visualise in our text data is the topics or key words that occur and their frequency across all of our responses — for example, we might want to see whether certain places come up a lot or not. This does happen with certain sentiments, feelings, and emotions too (provided they are not also in our list of stop words), but we can do more with our 'emotional' words than just count them. For now, let's stick with our topics. We could represent them in a bar chart after counting them up:

```
survey %>%
  select(time, resident, perfect_day) %>%
  unnest_tokens(word, perfect_day) %>%
  anti_join(stop_words) %>%
  count(word, sort = TRUE) %>%
  ggplot() +
  aes(x = word, y = n) +
  geom_col() +
  coord_flip()
```

Okay, this is way too much information to have on one graph, so what we might want to do is restrict our data to only the top 10 words. We can do that by using sort = TRUE (or arrange(desc(n))) and the using the slice() function to take the data from 1 to 10:

```
survey %>%
  select(time, resident, perfect_day) %>%
  unnest_tokens(word, perfect_day) %>%
  anti_join(stop_words) %>%
  count(word, sort = TRUE) %>%
  slice(1:10)
```

Now let's try plotting again, we can also reorder our word factor by the frequency the word comes up (n):

```
survey %>%
  select(time, resident, perfect_day) %>%
  unnest_tokens(word, perfect_day) %>%
  anti_join(stop_words) %>%
  count(word, sort = TRUE) %>%
  slice(1:10) %>%
  ggplot() +
  aes(x = fct_reorder(word, n), y = n) +
  geom_col() +
  coord_flip()
```

Can we spot any obvious problems with this? Probably! For example, a lot of these places are actually made up of two or three words (e.g. Rutland Arms, Red Brick Market, Nottingham House, Steamyard Cafe). We'll revisit how we might deal with this in a moment!

Before we move onto sentiment, let's use the ggwordcloud package to try making a wordcloud in R. We can usually fit more than 10 words in comfortably, so let's remove our slice() function at the same time:

```
survey %>%
  select(time, resident, perfect_day) %>%
  unnest_tokens(word, perfect_day) %>%
  anti_join(stop_words) %>%
  count(word, sort = TRUE) %>%
  ggplot() +
  aes(label = word, size = n) +
  geom_text_wordcloud()
```

You might find that your wordcloud is a little too big or too small. We can adjust the size of the wordcloud using the scale_size_area() function. For example:

```
survey %>%
  select(time, resident, perfect_day) %>%
  unnest_tokens(word, perfect_day) %>%
  anti_join(stop_words) %>%
  count(word, sort = TRUE) %>%
  ggplot() +
  aes(label = word, size = n) +
  geom_text_wordcloud() +
  scale_size_area(max_size = 10)
```

Visualising sentiment in text data

Quite often, we want to get a sense of how positive or negative a certain body of text is. If a body of text includes lots of positive words (pretty, nice, beautiful, wonderful) we might want to find a way of quantifying that and comparing it to bodies of text that have lots of negative words (miserable, grey, boring, etc.). Our 'perfect_day' question was more about topics and places, which do not have an inherent positive or negative emotional score attached to them, so let's switch now to how people felt about Sheffield before starting their studies (feel_before).

There are a very large number of dictionaries that researchers have created where they have attached **sentiment scores** to specific words. For example, let's take a look at the "AFINN" sentiment lexicon, so-named after the creator Finn Årep Nielson.

```
afinn <- get_sentiments("afinn")
sample_n(afinn, 20)
```

The second line of code above will randomly select twenty of the words in the dictionary to display, so you can compare some of the ones you got with your neighbour.

Now that we have this dictionary, we can **join** the sentiment scores onto the words from our survey responses so that we can use them in our data visualisation. In this case, we will use something called an **inner join** — this means that if our word cannot be found in the sentiment dictionary, or vice versa, we will remove that word entirely from our dataset. The by = "word"

```
survey %>%
  select(time, resident, feel_before) %>%
  unnest_tokens(word, feel_before) %>%
  anti_join(stop_words) %>%
  count(word, sort = TRUE) %>%
  inner_join(afinn, by = "word")
```

Now we have our sentiments, we can do a few different things. If we wanted to summarise our the sentiments of our responses as a whole, we could multiply the number of times a specific word comes up by the sentiment score it has (like how we adjusted for GDP in Week 4) and then visualise the result:

```
survey %>%
  select(time, resident, feel_before) %>%
  unnest_tokens(word, feel_before) %>%
  anti_join(stop_words) %>%
  count(word, sort = TRUE) %>%
  inner_join(afinn, by = "word") %>%
  mutate(
    sentiment = n*value
  ) %>%
  arrange(desc(sentiment)) %>%
  slice(1:10) %>%
  ggplot() +
  aes(y = sentiment, x = fct_reorder(word, sentiment)) +
  geom_col() +
  coord_flip()
```

We could also make a wordcloud again, but this time we could make the size of the word be relative to the number of times it came up and the colour of the word be relative to the sentiment.

```
survey %>%
  select(time, resident, feel_before) %>%
  unnest_tokens(word, feel_before) %>%
  anti_join(stop_words) %>%
  count(word, sort = TRUE) %>%
  inner_join(afinn, by = "word") %>%
  ggplot() +
  aes(label = word, size = n, colour = value) +
  geom_text_wordcloud() +
  scale_size_area(max_size = 10) +
  scale_colour_gradient(low = "navy", high = "coral")
```

Bigrams and ngrams

One of the problems we mentioned earlier is that we could very well have meaningful topics that are two or three words put together, e.g. “Peak District”, but these would be disconnected in our treatment of the text. Combinations of two words are called ‘bigrams’, and combinations of three words are called ‘trigrams’. Together, these are called ngrams. We could imagine that we want to know about pairs or triads of words that come up together often when analysis data.

We can prepare our data into bigrams or trigrams by modifying our unnest_tokens argument. Let’s go back to our perfect_day variable for this:

```
survey %>%
  select(time, resident, perfect_day) %>%
  unnest_tokens(word, perfect_day, token = "ngrams", n = 2)
```

However, now we have a bit of a problem — how do we get rid of our stop words when all of a sudden none of our cells contain single words? One approach is to split our word column up into two separate columns, word1 and word2 of our bigrams. We can use the separate() function in order to do this.

```
survey %>%
  select(time, resident, perfect_day) %>%
  unnest_tokens(word, perfect_day, token = "ngrams", n = 2) %>%
  separate(word, into = c("word1", "word2"), sep = " ")
```

In the separate function we have three arguments: the first is the column we want to split up, the second tells R what we want the new columns to be called, and the third tells R where we want to split the columns (where there is a space).

Now we can get rid of our stop words by using the filter function from previous weeks:

```
survey %>%
  select(time, resident, perfect_day) %>%
  unnest_tokens(word, perfect_day, token = "ngrams", n = 2) %>%
  separate(word, into = c("word1", "word2"), sep = " ") %>%
  filter(
    !word1 %in% stop_words$word,
    !word2 %in% stop_words$word
  )
```

Let's recap what filter() is doing here — we have two conditions. The first one is !word1 %in% stop_words$word. word1 %in% stop_words$word means any of the words in our word1 column that show up in the column of stop words that are saved in our stop_words$word column. The ! at the start of the statement means "NOT", so, in other words, we want to filter all of the rows that do NOT have words in the word1 column that show up in our list of stop words. Then, we want to do the same thing for the word2 column.

Let's count up our results and see if there are any interesting differences in our word cloud:

```
survey %>%
  select(time, resident, perfect_day) %>%
  unnest_tokens(word, perfect_day, token = "ngrams", n = 2) %>%
  separate(word, into = c("word1", "word2"), sep = " ") %>%
  filter(
    !word1 %in% stop_words$word,
    !word2 %in% stop_words$word
  ) %>%
  count(word1, word2, sort = TRUE) %>%
  ggplot() +
  aes(label = paste(word1, word2), size = n) +
  geom_text_wordcloud()
```

Shaping our wordcloud

One thing that I haven't mentioned so far is the shape of the wordcloud. The ggwordcloud package does have a number of built in shapes that we can use. For example, in our previous wordcloud, we could change the shape to a diamond by adding shape = "diamond" to our geom_text_wordcloud() function. The built in shapes are: "circle", "cardioid", "diamond", "square", "triangle-forward", "triangle-upright", "pentagon", and "star".

```
survey %>%
  select(time, resident, perfect_day) %>%
  unnest_tokens(word, perfect_day, token = "ngrams", n = 2) %>%
  separate(word, into = c("word1", "word2"), sep = " ") %>%
  filter(
    !word1 %in% stop_words$word,
    !word2 %in% stop_words$word
  ) %>%
  count(word1, word2, sort = TRUE) %>%
  ggplot() +
  aes(label = paste(word1, word2), size = n) +
  geom_text_wordcloud(shape = "diamond")
```

We may need to skip over this part in the interest of time. We can also use images of with transparent backgrounds in order to force our wordcloud into any shape we like. The ggwordcloud packages comes with some of these built in, for example, if we wanted to make our wordcloud fit the shape of a heart (as much as possible, at least!), we can do this by providing the function with information about the image of a heart with a transparent background outside the heart.

```
survey %>%
  select(time, resident, perfect_day) %>%
  unnest_tokens(word, perfect_day, token = "ngrams", n = 2) %>%
  separate(word, into = c("word1", "word2"), sep = " ") %>%
  filter(
    !word1 %in% stop_words$word,
    !word2 %in% stop_words$word
  ) %>%
  count(word1, word2, sort = TRUE) %>%
  ggplot() +
  aes(label = paste(word1, word2), size = n) +
  geom_text_wordcloud_area(mask = png::readPNG(system.file("extdata/hearth.png",
      package = "ggwordcloud", mustWork = TRUE
    )), rm_outside = TRUE)
```

Back to sentiment and bigrams

Another reason we may want to use bigrams rather than single words is to detect problems with our sentiment scoring. For example, imagine that we had a bunch of responses that said things like "I am not happy" or "I never enjoy Sheffield" — our data processing would just delete the "not"s and "never"s and keep the "happy" and "enjoy", rating them very positively! We probably don't want that to happen. We can use bigrams to detect these "negating words".

First, let's create a list of negation words, that is, words that modify the word after it to mean the opposite.

```
negation_words <- c("no", "not", "don't", "never", "without")
```

You can probably think of more examples. Now let's go to how people felt after starting their studies, the feel_now variable, and prepare our data as bigrams. We can check whether we have any cases like those described:

```
survey %>%
  select(time, resident, feel_now) %>%
  unnest_tokens(word, feel_now, token = "ngrams", n = 2) %>%
  separate(word, into = c("word1", "word2"), sep = " ") %>%
  filter(word1 %in% negation_words)
```

Since we have at least one, we might want to reverse the sentiment score after adding it. Let's add sentiment scores based on the second word in our bigrams

```
survey %>%
  select(time, resident, feel_now) %>%
  unnest_tokens(word, feel_now, token = "ngrams", n = 2) %>%
  separate(word, into = c("word1", "word2"), sep = " ") %>%
  inner_join(afinn, by = c("word2"="word"))
```

Note that here we need to specify explicitly that it is the second column, word2, that we want to add our sentiment scores to.

Now let's reverse the sentiment scores for bigrams where the first word includes a negation; we can reverse a number by multiplying it by minus 1. We are going to use mutate for this, but with a slightly more complicated statement. Let's add on the code and then I'll explain it.

```
survey %>%
  select(time, resident, feel_now) %>%
  unnest_tokens(word, feel_now, token = "ngrams", n = 2) %>%
  separate(word, into = c("word1", "word2"), sep = " ") %>%
  inner_join(afinn, by = c("word2"="word")) %>%
  mutate(
    value = ifelse(word1 %in% negation_words,
                   yes = value*-1,
                   no = value)
  )
```

So, what are we doing here:

- `value =` we are overwriting the value variable with whatever the result of the things on the right-hand side of the equals is
- `ifelse(` we are using a function called ifelse, which required three arguments: the first argument is a statement that evaluates to true or false for each of our rows, in this case, whether the value in the word1 column is the same as any of the words in our negation_words vector, then…
- `yes = value*-1` meaning, if our value in the word1 column *is* in our negation list, we are going to take the original value from the value column and multiply it by -1
- `no = value` meaning, if our word1 is not in our list of negation words, we are just going to leave the value alone.

Finally, we want to modify our word2 column so that our negated words are turned into bigrams with their negation. We can do that by adding a second change in our mutate function, this time modifying the word2 column.

```
survey %>%
  select(time, resident, feel_now) %>%
  unnest_tokens(word, feel_now, token = "ngrams", n = 2) %>%
  separate(word, into = c("word1", "word2"), sep = " ") %>%
  inner_join(afinn, by = c("word2"="word")) %>%
  mutate(
    value = ifelse(word1 %in% negation_words,
                   yes = value*-1,
                   no = value),
    word2 = ifelse(word1 %in% negation_words,
                   yes = paste(word1, word2),
                   no = word2
                   )
  )
```

So, similar to before, but this time we are combining word1 and word2 together using the paste() function **if and only if** the corresponding word1 is in our list of negation words. Let's finish this off by dropping our stop words, multiplying our sentiment by the number of times each word/bigram appears, taking the top twenty words, and then creating our sentiment bar plot:

```
survey %>%
  select(time, resident, feel_now) %>%
  unnest_tokens(word, feel_now, token = "ngrams", n = 2) %>%
  separate(word, into = c("word1", "word2"), sep = " ") %>%
  inner_join(afinn, by = c("word2"="word")) %>%
  mutate(
    value = ifelse(word1 %in% negation_words,
                   yes = value*-1,
                   no = value),
    word2 = ifelse(word1 %in% negation_words,
                   yes = paste(word1, word2),
                   no = word2
                   )
  ) %>%
  count(word2, value) %>%
  anti_join(stop_words, by = c("word2" = "word")) %>%
  mutate(sentiment = value*n) %>%
  arrange(desc(sentiment)) %>%
  slice(1:20) %>%
  ggplot() +
  aes(x = sentiment, y = fct_reorder(word2, sentiment)) +
  geom_col()
```

We might not get time for this, but I'll try and demo it in class

At the beginning of the session, we loaded a package called LocalLLM. Back before large language models (LLMs) were just synonymous with chatbots, we thought about them by their original purpose: to generate realistic-sounding responses to text based on probability. This is made possible by training models on vast quantities of real text, where the underlying structures and relationships between words could inform the probability of certain responses.

The LocalLLM package in R allows us to download a quite large (2GB) open LLM to our own computer and then use it to transform text locally, using our own computer's processing power (meaning no big data centres, no sending our data off to big companies who run the websites that host the chatbots).

Before we can use it, we need to install some of the programming tools that it uses in the background (tools written in a very fast language called C++). We can do this by running the following:

```
install_localLLM()
```

This **may or may not work** depending on whether you are using a university computer or your own laptop, and how much space your own laptop has.

The first time we use LocalLLM, it will download a copy of a fairly modestly sized open LLM (Llama) and use this to respond to our queries. This will take some time — 2GB will take several minutes even on a fast computer. The speed at which you get a result to your query will depend on how powerful your computer is. Let's try it now

```
response <- quick_llama("How can good data visualisation skills help me get a job?")
cat(response)
```

In this case, cat() just makes the output display in a more user-friendly way (without lots of formatting junk). We can adjust the length of the output by changing the max_tokens argument — we just learned what a token was earlier!

```
response <- quick_llama("How can good data visualisation skills help me get a job?",
max_tokens = 150)
cat(response)
```

We can also do other things, like set the system prompt which gives the LLM context to drive its responses for every single request (we could make multiple in a single line of code, we'll see that in a moment)

```
response <- quick_llama("How can good data visualisation skills help me get a job?",
max_tokens = 100, system_prompt = "Give a bad answer on purpose.")
cat(response)
```

We can also change the temperature of the model — this is basically how "spicy" and creative the responses are. We get more unpredictable responses the closer the temperature is to 1, and more predictable and less creative responses when it is closer to 0. By default, the temperature is set to 0. We can also set a seed to ensure we get the same result every time.

Okay — how does this help us? Remember back when we tried to identify activities and places in Sheffield for the perfect day and we got a lot of results that weren't quite right, for example, we were picking up "Red Brick" and "Brick Market" but not "Red Brick Market"? One thing we could do is ask our local model to identify and summarise activities or events that people recommend in their responses.

```
response <- quick_llama(survey$perfect_day,
                        max_tokens = 50,
                        system_prompt = "You will be provided with survey responses a
bout activities and places people recommend in Sheffield. For each response, extract
the names of the places (other than Sheffield itself) and/or the activities in the re
sponse and separate multiple activities/places with commas. Never produce any additio
nal information, only the names of places/activities. Ensure all names/places are in
title case.", seed = 72, temperature = 0)
paste(response)
```

So, what are we doing here? Instead of writing out a single prompt, we are providing the model with all of our ‘perfect_day’ responses as separate prompts. We are then using the system_prompt argument to provide context on what we want the response to be. Essentially, our LLM is able to pick out probable place and activity names based on its training on grammatical and sentence structure. We might get some nonsense responses, which is part of the risk of these models!

Now we have some responses we can add them back onto our dataset.

```
survey %>%
  mutate(
    recommendations = paste(response)
  ) %>%
  select(resident, recommendations)
```

Now we can unnest our recommendations based on where there are commas, using a slightly modified unnest_tokens() function

```
survey %>%
  mutate(
    recommendations = paste(response)
  ) %>%
  select(resident, recommendations) %>%
  unnest_tokens(word, recommendations,
                token = "regex", pattern = ",",
                to_lower = FALSE)
```

Lastly, this could use a little tidying up by removing the extra spaces. Then we’ll be ready to plot. We can do this using mutate() and the str_trim() function.

```
survey %>%
  mutate(
    recommendations = paste(response)
  ) %>%
  select(resident, recommendations) %>%
  unnest_tokens(word, recommendations,
                token = "regex", pattern = ",",
                to_lower = FALSE) %>%
  mutate(
    word = str_trim(word)
  )
```

Now let’s finally make our word cloud of recommended places and activities, coloured by whether someone lived in Sheffield before their studies or not.

```
survey %>%
  mutate(
    recommendations = paste(response)
  ) %>%
  select(resident, recommendations) %>%
  unnest_tokens(word, recommendations,
                token = "regex", pattern = ",",
                to_lower = FALSE) %>%
  mutate(
    word = str_trim(word)
  ) %>%
  count(resident, word) %>%
  ggplot() +
  aes(label = word, size = n, colour = resident) +
  geom_text_wordcloud() +
  scale_size_area(max_size = 10)
```

Not bad!

Task

We did a lot of new stuff today which will help us with our data tidying in future weeks, and with building up our skills (e.g. building a data pipeline from Google Forms, to Sheets, into R, and then processing with a local LLM before creating visualisation — that's a pretty good example for an employer!) So, I have one easier task for you to try, and one harder. I don't mind if you only do the easier one or just the harder one.

1. Create a topic wordcloud or bar plot for the know_before survey question using both the single word method of tokenization and the bigram method, just for people who did not live in Sheffield already before starting their course. Do you get similar results? Does this help us better understand what people know about Sheffield before they start their course?

2. **More difficult task** At the start, we installed and loaded the gutenbergr package. This is a package for downloading the full text of public domain works of fiction. For example, if you run `frankenstein <- gutenberg_download(84)` , you will download a copy of Mary Shelley's Frankenstein and save it to the frankenstein object. With this data, I would like you to try: 1) creating a new variable that represents the line of the text of the book (you will need to use mutate and the row_number() function) and then a second variable which represents what percentage range of the book this represents (the cut_interval function is the key to this); 2) tokenize the text of the book and then add sentiment scores to each token; 3) summarise the sentiment score for each percentage of lines of the book using group_by and summarise with your earlier created variables; 4) create a bar plot that shows what the sentiment of the text is in the book changes as the story progresses. This is a tricky task so don't worry if you can't do it! The resulting plot is very interesting though, so be sure to check the solutions next week!